Flyweight

Flyweight · Share glyph styles across a million characters · Build with Waqar

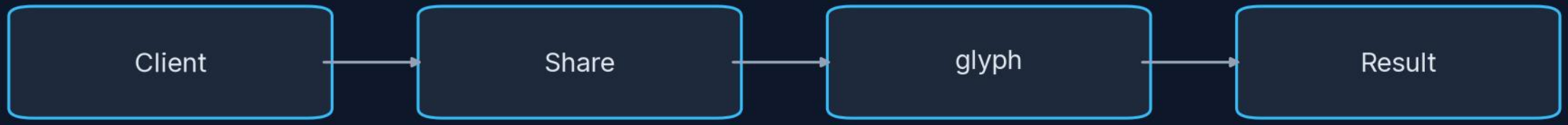
Today's Hook

- Problem: Share glyph styles across a million characters
- Pattern: Flyweight — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Share glyph styles across a million characters
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Flyweight: Share glyph styles across a million characters

Real-World Scenario

- Running example: Share glyph styles across a million characters
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```java
package com.buildwithwaqar.patterns.gof.structural.flyweight.problem;

import java.util.*;

public final class NaiveGlyphs {
  public static int render(int n){
    int styles=0; for(int i=0;i<n;i++) styles++; // new style each glyph
    return styles;
  }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```
package com.buildwithwaqar.patterns.gof.structural.flyweight.problem;

import java.util.*;
import java.util.concurrent.*;
import java.util.function.*;

final class GlyphStyle {
  private final String font; private final int size;
  GlyphStyle(String font, int size){ this.font=font; this.size=size; }
  String key(){ return font+":"+size; }
}
final class GlyphFactory {
  private final Map<String, GlyphStyle> cache = new HashMap<>();
  GlyphStyle style(String font, int size){
    return cache.computeIfAbsent(font+":"+size, k -> new GlyphStyle(font, size));
  }
  int cached(){ return cache.size(); }
}
final class Glyph {
  private final char c; private final GlyphStyle style;
  Glyph(char c, GlyphStyle style){ this.c=c; this.style=style; }
}
```

Runnable main

```
package com.buildwithwaqar.patterns.gof.structural.flyweight.problem;

import java.util.*;
import java.util.concurrent.*;

public final class ProblemDemo {
  public static void main(String[] args) {
    System.out.println("=== Flyweight: share glyph styles ===");
    System.out.println("--- naive ---");
System.out.println("stylesCreated="+NaiveGlyphs.render(1000));
System.out.println("--- flyweight ---");
var f=new GlyphFactory();
for (int i=0;i<1000;i++) new Glyph((char)('a'+(i%26)), f.style("Inter", 12));
System.out.println("cachedStyles="+f.cached());
  }
}
```

Walkthrough Output

```
=== Flyweight: share glyph styles ===
--- naive ---
stylesCreated=1000
--- flyweight ---
cachedStyles=1
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Flyweight.

Related Patterns

- Contrast Flyweight with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Flyweight → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.